Software Implementation and Testing Document

For

Group <RuneScape Web App>

Version 2.0

Authors:
Agustin Camus
Colton Bregoff
Miles Burkart
Ricardo Furst
William Kilduff

1. Programming Languages (5 points)

RuneReport is a static, client-side web application with no backend, so the whole project is written in the three standard languages of the web:

- **HTML -** Defines the structure of every page. Each page contains the shared navigation bar, the page title, and the page-specific controls such as the Browse search box. A `<template>` element

defines one buy/sell condition row, which JavaScript then copies each time the user adds a condition.

- **CSS -** Handles all styling: the dark-brown and gold theme, the navigation bar, and the flexbox layouts for the Browse controls and item list. It also contains the keyframe animations for the RuneScape-style text effects.
- **JavaScript -** Provides all client-side behavior. Each page has its own script that fetches market data from the API with `fetch()`, then builds the page from it. This covers the search suggestions, the Best Deals ranking, Browse sorting and pagination, live item stats with the 2% tax margin, the condition builder, and saving reports to local storage. The shared navbar (components/navbar.js) builds the same links on every page and highlights the current one. In increment 3, JavaScript will also run the strategy engine.

We chose these languages because the project is a client-side web application that needs no server, which keeps development and deployment simple. JavaScript lets the pages work with market data and run the strategy tests directly in the browser, and it is the standard way to do so. Several team members were new to JavaScript, but we chose it for the project's needs and are learning it as we go.

2. Platforms, APIs, Databases, and other technologies used (5 points)

Technology	Type	Use in the project
OSRS Wiki Real-time Prices API	External API	Source of Old School RuneScape Grand Exchange item and historical price data. It feeds the Browse page item list, the item pages, and the backtesting engine that replays user-defined strategies against historical prices
OSRS Wiki image server	External assets	Item icons on the browse, item, and report pages are loaded by the icon file name that /mapping returns.
OsFont (os-font.js / os-font.css)	Front-end library	A small js and CSS helper that applies RuneScape chat-style text effects (colors, flash, glow, wave, shake, scroll, slide) to paragraphs with prefixes such as flash1:wave:. It is loaded on every page and runs with OsFont.compile(). It is third-party code included in the repository under fonts.

RuneScape Chat Font	Font asset	Provides the in-game look for headings and effect text. It is packaged in fonts/font/ as a @font-face kit and used by the page title and OsFont effects.
GitHub (7Limes/CEN4090L-Project)	Version control/hosting	Hosts the source code and the project documents. Instructor and TAs have access to this public repository.
GitHub Issues	Project Tracking	Tracks features and bugs, grouped by increment. Every pull request is linked to an issue.
Discord	Communication	Team calls and coordination outside of class meetings.
Web browser (client)	Platform	The application runs entirely in the user's browser. No server-side code or user accounts exist; all users are guests.
Browse local storage (Web storage API)	Browser Platform	Stores the saved reports on the user's device.
Browse Fetch API	Browser Platform	Requests JSON from the Prices API. Each page makes its requests once on load, in parallel where it needs several.

Databases: This project uses no database. All data is retrieved from the OSRS Wiki Real-time Prices API on demand.
Saved reports (shown on the "Saved Reports" page) are stored in the browser's local storage, the client side only, under the key savedReports. This is done as a JSON list ordered newest first. Each entry holds:

- `id`: the time it was saved, which is also unique.
- `itemId` and `itemName`
- `icon`: the item's icon URL
- `saveDate`
- `html`: a snapshot of the report

The Saved Reports page reads this list, shows 10 reports per page, and links each one to the report page with the `?report=<id>`. The report page then redraws the snapshot exactly as saved, and they stay on that device and browser only.

3. Execution-based Functional Testing (10 points)

We performed functional testing throughout development by testing our individual features to make sure they worked as expected. Each team member tested their own changes before submitting pull requests. We also individually reviewed the new features implemented to help identify bugs, html errors, or logic errors. We did this by checking our features against the project's functional and non-functional requirements. Testing included verifying that item searches returned the correct results, item pages displayed data from the OSRS API, and the Browse page correctly filtered and sorted items. We also checked that item information updated using API data and that saved reports persisted in browser storage. We went over requirements such as data accuracy, API usage, and reliability when testing the system. The Strategy Backtester and actual report generation have not yet been completed, so those features have not undergone testing.

4. Execution-based Non-Functional Testing (10 points)

We performed non-functional testing by looking through our non-functional requirements and testing relevant elements in our application. We tested for performance by timing multiple API requests and ensuring that the client received a response within three seconds. We ensured API compliance by structuring our app around large infrequent requests rather than small requests which occur more often. We tested for reliability by simulating a realistic API outage and ensuring that our app responded properly.

5. Non-Execution-based Testing (10 points)

We had an accountability system as we made changes and made sure anything that was previously committed was actually functional. Moving the project forward with a mistake would set us back a meaningful amount of time, which we did not have. When making a new branch for a new issue we took the previous commit and ensured every feature worked as intended before moving forward.